

Mobile and AR Access to PLM

Viewing 3D models and work instructions on the shop floor - Coding and Circuit

1. Project overview

Operators on the shop floor need the **current, released** 3D model and work instructions without walking to a PC. This project connects a low-cost **RFID station reader** (ESP32) to a **backend service** that pulls the work order and CAD model from the PLM system, and a **mobile web app** that shows the 3D model, the step-by-step instructions, and an **AR view** (WebXR on Android, Quick Look on iPhone/iPad).

Flow: operator taps the work-order RFID tag on the reader → ESP32 sends the tag UID to the server → server looks up the work order in PLM → the tablet/phone at that station (already open on its station page) updates automatically to the 3D model and step 1 → the operator advances steps on the screen or with the physical button.

2. System architecture

Layer	Component	Role
Edge hardware	ESP32 + MFRC522 + OLED + LEDs + button	Identifies work order by RFID tag; gives local feedback; physical "Next step" button
Backend	Python Flask (app.py)	Holds per-station state, adapts to PLM REST API, serves viewer app and models
PLM	Windchill / Teamcenter / Arena / other	Source of released work order, BOM and CAD (exported as glTF .glb / .usdz)
Mobile app	HTML + JS + <model-viewer>	Interactive 3D view, AR placement, instruction steps

Model preparation: export released CAD from PLM (or a neutral STEP file) and convert to **.glb** (Android/web) and **.usdz** (iOS AR). Tools: Blender, FreeCAD, Autodesk Forge/APS, or Siemens/PTC visualization converters. Keep models under ~10 MB; decimate meshes for phones.

3. Bill of materials (station reader)

Qty	Part	Notes
1	ESP32 DevKit V1 (30-pin)	Wi-Fi + 3.3V logic
1	MFRC522 RFID module (13.56 MHz)	With MIFARE cards/tags for work orders
1	SSD1306 0.96" OLED, I2C, 128x64	Address 0x3C
2	5 mm LED (green, red)	Status feedback
2	220 ohm resistor	One per LED
1	Momentary push button	Next-step input
1	Breadboard / perfboard + jumpers; 5V USB supply	Use an enclosure for the shop floor

4. Circuit

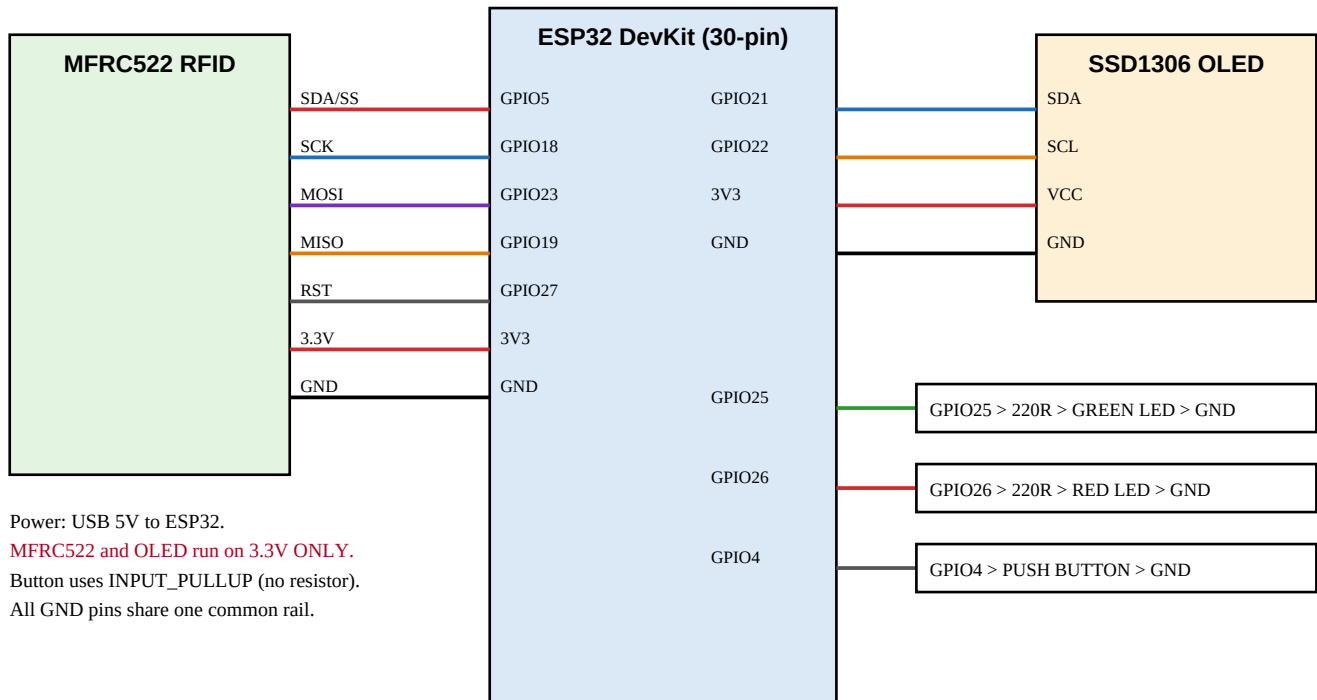


Figure 1 - Shop-floor station reader wiring

Device pin	ESP32 pin	Notes
MFRC522 SDA (SS)	GPIO5	SPI chip select
MFRC522 SCK	GPIO18	VSPI clock
MFRC522 MOSI	GPIO23	VSPI MOSI
MFRC522 MISO	GPIO19	VSPI MISO
MFRC522 RST	GPIO27	Reset
MFRC522 3.3V / GND	3V3 / GND	3.3 V only - 5 V damages the module
OLED SDA / SCL	GPIO21 / GPIO22	I2C bus
OLED VCC / GND	3V3 / GND	
Green LED anode (via 220R)	GPIO25	Cathode to GND
Red LED anode (via 220R)	GPIO26	Cathode to GND
Push button	GPIO4 to GND	Internal pull-up enabled in firmware

5. Firmware - ESP32 (Arduino IDE / PlatformIO)

Install the board package "esp32 by Espressif", then libraries: **MFRC522**, **Adafruit SSD1306**, **Adafruit GFX**, **ArduinoJson** (v7). Edit the CONFIG block (Wi-Fi, server IP, API key, station ID) and upload.

```
// station_reader.ino - ESP32 + MFRC522 + SSD1306
// Libraries: MFRC522 (miguelpalboa), Adafruit SSD1306, Adafruit GFX, ArduinoJson 7
#include <WiFi.h>
#include <HTTPClient.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <ArduinoJson.h>

// ---- CONFIG ----
const char* WIFI_SSID = "SHOP_WIFI";
const char* WIFI_PASS = "change-me";
const char* SERVER = "http://192.168.1.50:5000"; // Flask server
const char* API_KEY = "change-me"; // must match STATION_KEY
const char* STATION_ID = "ST01";

// ---- PINS ----
#define SS_PIN 5
#define RST_PIN 27
#define LED_OK 25
#define LED_ERR 26
#define BTN_NEXT 4

MFRC522 rfid(SS_PIN, RST_PIN);
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
unsigned long lastBtn = 0;

void show(const String& l1, const String& l2 = "", const String& l3 = "") {
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0, 0); oled.println(STATION_ID);
  oled.setCursor(0, 16); oled.println(l1);
  oled.setCursor(0, 32); oled.println(l2);
  oled.setCursor(0, 48); oled.println(l3);
  oled.display();
}

void flash(int pin) {
  digitalWrite(pin, HIGH); delay(300); digitalWrite(pin, LOW);
}

void connectWiFi() {
  show("Connecting WiFi...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  unsigned long t0 = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - t0 < 20000) delay(250);
  if (WiFi.status() == WL_CONNECTED) show("WiFi OK", WiFi.localIP().toString());
  else show("WiFi FAILED");
}

// POST a JSON body, return parsed response in `out`. Returns HTTP code.
int postJson(const String& path, const String& body, JsonDocument& out) {
  if (WiFi.status() != WL_CONNECTED) connectWiFi();
  HTTPClient http;
  http.begin(String(SERVER) + path);
  http.addHeader("Content-Type", "application/json");
  http.addHeader("X-Api-Key", API_KEY);
  http.setTimeout(5000);
  int code = http.POST(body);
  if (code > 0) deserializeJson(out, http.getString());
  http.end();
  return code;
}
```

```

String uidToString(MFRC522::Uid& u) {
    String s;
    for (byte i = 0; i < u.size; i++) {
        if (u.uidByte[i] < 0x10) s += "0";
        s += String(u.uidByte[i], HEX);
    }
    s.toUpperCase();
    return s;
}

void handleScan(const String& uid) {
    JsonDocument req, res;
    req["station"] = STATION_ID;
    req["uid"] = uid;
    String body; serializeJson(req, body);
    int code = postJson("/api/scan", body, res);
    if (code == 200 && res["ok"] == true) {
        show(res["order"].as<String>(), res["part"].as<String>(), "Open app on tablet");
        flash(LED_OK);
    } else {
        show("Unknown tag", uid, "HTTP " + String(code));
        flash(LED_ERR);
    }
}

void handleNext() {
    JsonDocument req, res;
    String body; serializeJson(req, body);
    int code = postJson("/api/station/" + String(STATION_ID) + "/next", body, res);
    if (code == 200) {
        show(res["order"].as<String>(),
            "Step " + String(res["step"].as<int>() + 1) + "/" + String(res["total"].as<int>()));
        flash(LED_OK);
    } else flash(LED_ERR);
}

void setup() {
    Serial.begin(115200);
    pinMode(LED_OK, OUTPUT); pinMode(LED_ERR, OUTPUT);
    pinMode(BTN_NEXT, INPUT_PULLUP);
    Wire.begin(21, 22);
    if (!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) Serial.println("OLED not found");
    SPI.begin(18, 19, 23, SS_PIN); // SCK, MISO, MOSI, SS
    rfid.PCD_Init();
    connectWiFi();
    show("Ready", "Scan work-order tag");
}

void loop() {
    if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
        String uid = uidToString(rfid.uid);
        Serial.println("UID " + uid);
        handleScan(uid);
        rfid.PICC_HaltA();
        rfid.PCD_StopCrypto1();
    }
    if (digitalRead(BTN_NEXT) == LOW && millis() - lastBtn > 600) {
        lastBtn = millis();
        handleNext();
    }
    delay(20);
}

```

6. Backend - Python Flask (app.py)

Run: `pip install flask requests` then `python app.py`. Set `PLM_URL` and `PLM_TOKEN` to use a real PLM; the `lookup_work_order()` function is the single place to adapt to your PLM's REST API.

```
# app.py - Flask backend: station state + PLM adapter + static AR viewer
# pip install flask requests
import os, time, json
import requests
from flask import Flask, jsonify, request, send_from_directory, abort

app = Flask(__name__, static_folder="static", static_url_path="")
STATION_KEY = os.environ.get("STATION_KEY", "change-me")
PLM_URL = os.environ.get("PLM_URL") # e.g. https://plm.example.com/api
PLM_TOKEN = os.environ.get("PLM_TOKEN", "")

# ---- Local demo data (used when PLM_URL is not set) -----
# tag UID -> work order. "model" is a .glb (and optional .usdz for iOS) in /static/models
DEMO = {
    "04A1B2C3": {
        "order": "W0-1042",
        "part": "Gearbox GB-200 Rev C",
        "model": "/models/gearbox.glb",
        "ios_model": "/models/gearbox.usdz",
        "steps": [
            {"title": "Prepare housing", "text": "Clean housing bore. Check for burrs.", "camera": "0deg 75deg 2.5m"},
            {"title": "Install input shaft", "text": "Press bearing onto shaft. Torque nut to 25 Nm.", "camera": "90deg 70deg"},
            {"title": "Fit cover", "text": "Apply sealant. Fasten 6x M6 bolts in star pattern, 10 Nm.", "camera": "180deg 45deg"},
            {"title": "Final inspection", "text": "Rotate shaft by hand. Record result in QA form.", "camera": "0deg 90deg 3m"}
        ]
    }
}

STATIONS = {} # station_id -> {"order":..., "step": int, "ts": float}

def lookup_work_order(uid):
    """Return work-order dict for a tag UID, from PLM if configured, else demo data."""
    if PLM_URL:
        # ADAPT to your PLM (Windchill, Teamcenter, Arena, Fusion Manage, ...).
        # Expected result shape = same as DEMO entries.
        r = requests.get(f"{PLM_URL}/work-orders/by-tag/{uid}",
                        headers={"Authorization": f"Bearer {PLM_TOKEN}"}, timeout=5)
        if r.status_code == 200:
            return r.json()
        return None
    return DEMO.get(uid)

def need_key():
    if request.headers.get("X-API-Key") != STATION_KEY:
        abort(401)

def snapshot(station):
    s = STATIONS.get(station)
    if not s:
        return None
    wo = s["wo"]
    return {"order": wo["order"], "part": wo["part"], "model": wo["model"],
            "ios_model": wo.get("ios_model"), "step": s["step"],
            "total": len(wo["steps"]), "current": wo["steps"][s["step"]],
            "steps": [x["title"] for x in wo["steps"]], "ts": s["ts"]}

@app.post("/api/scan")
def scan():
    need_key()
    data = request.get_json(force=True)
    wo = lookup_work_order(data.get("uid", "").upper())
    if not wo:
        return jsonify(ok=False, error="unknown tag"), 404
    STATIONS[data["station"]] = {"wo": wo, "step": 0, "ts": time.time()}
    return jsonify(ok=True, order=wo["order"], part=wo["part"])

@app.post("/api/station/<sid>/next")
```

```

def next_step(sid):
    need_key()
    s = STATIONS.get(sid)
    if not s:
        return jsonify(ok=False, error="no active order"), 404
    s["step"] = min(s["step"] + 1, len(s["wo"]["steps"]) - 1)
    s["ts"] = time.time()
    return jsonify(ok=True, order=s["wo"]["order"], step=s["step"], total=len(s["wo"]["steps"]))

# Viewer endpoints (no key: read-only; put behind your plant VPN / SSO in production)
@app.get("/api/station/<sid>/current")
def current(sid):
    snap = snapshot(sid)
    return (jsonify(snap), 200) if snap else (jsonify(error="idle"), 204)

@app.post("/api/station/<sid>/step/<int:n>")
def set_step(sid, n):
    s = STATIONS.get(sid) or abort(404)
    s["step"] = max(0, min(n, len(s["wo"]["steps"]) - 1))
    s["ts"] = time.time()
    return jsonify(ok=True)

@app.get("/")
def index():
    return send_from_directory("static", "index.html")

if __name__ == "__main__":
    # AR (WebXR/Scene Viewer) needs HTTPS on phones: use a reverse proxy
    # (nginx/Caddy) or: flask run --cert=adhoc (needs `pip install pyopenssl`)
    app.run(host="0.0.0.0", port=5000)

```

7. Mobile / AR web app (static/index.html)

Folder layout: app.py, static/index.html, static/models/gearbox.glb, static/models/gearbox.usdz. Open <https://SERVER/?station=ST01> on the tablet or phone at that station.

```
<!-- static/index.html : mobile web app, open as https://SERVER/?station=ST01 -->
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
<title>Shop-floor PLM Viewer</title>
<script type="module"
  src="https://cdn.jsdelivr.net/npm/@google/model-viewer@3.5.0/dist/model-viewer.min.js"></script>
<style>
:root { --bg:#10151c; --fg:#eef2f6; --card:#1b232e; --acc:#2f9e44; }
* { box-sizing: border-box; }
body { margin:0; font-family: system-ui, sans-serif; background:var(--bg); color:var(--fg);
  display:flex; flex-direction:column; height:100vh; }
header { padding:10px 14px; background:var(--card); }
header b { font-size:1.1rem; }
model-viewer { flex:1; width:100%; background:#2a3442; --poster-color:transparent; }
#ar-btn { position:absolute; right:12px; bottom:12px; padding:10px 14px; border:0;
  border-radius:8px; background:var(--acc); color:#fff; font-weight:700; }
#panel { background:var(--card); padding:12px 14px; }
#panel h3 { margin:0 0 4px; }
#panel p { margin:0 0 10px; font-size:1.05rem; line-height:1.35; }
.row { display:flex; gap:10px; }
.row button { flex:1; padding:14px; font-size:1rem; border:0; border-radius:8px;
  background:#3a4658; color:#fff; }
.row button.main { background:var(--acc); }
#idle { padding:40px 20px; text-align:center; font-size:1.2rem; }
</style>
</head>
<body>
<header><b id="wo">Waiting for work order...</b><br><small id="part"></small></header>
<div id="idle">Scan a work-order tag at the station reader.</div>

<model-viewer id="mv" style="display:none" camera-controls touch-action="pan-y"
  ar ar-modes="webxr scene-viewer quick-look" ar-scale="fixed"
  shadow-intensity="1" exposure="1" alt="3D model of current assembly">
  <button slot="ar-button" id="ar-btn">View in AR</button>
</model-viewer>

<div id="panel" style="display:none">
  <h3 id="title"></h3>
  <p id="text"></p>
  <div class="row">
    <button id="prev">&#9664; Back</button>
    <span id="count" style="align-self:center"></span>
    <button id="next" class="main">Next &#9654;</button>
  </div>
</div>

<script>
const station = new URLSearchParams(location.search).get("station") || "ST01";
const $ = id => document.getElementById(id);
let last = 0, step = 0;

async function poll() {
  try {
    const r = await fetch(`/api/station/${station}/current`);
    if (r.status === 200) {
      const d = await r.json();
      if (d.ts !== last) { last = d.ts; render(d); }
    }
  } catch (e) { /* offline: keep last view */ }
}

function render(d) {
```

```

step = d.step;
$("#wo").textContent = d.order;
$("#part").textContent = d.part;
$("#idle").style.display = "none";
$("#mv").style.display = "block";
$("#panel").style.display = "block";
const mv = $("#mv");
if (mv.getAttribute("src") !== d.model) {
  mv.setAttribute("src", d.model);
  if (d.ios_model) mv.setAttribute("ios-src", d.ios_model);
}
mv.setAttribute("camera-orbit", d.current.camera || "0deg 75deg 2.5m");
$("#title").textContent = `Step ${d.step + 1}: ${d.current.title}`;
$("#text").textContent = d.current.text;
$("#count").textContent = `${d.step + 1} / ${d.total}`;
$("#prev").disabled = d.step === 0;
$("#next").disabled = d.step === d.total - 1;
window.TOTAL = d.total;
}

async function go(n) {
  await fetch(`/api/station/${station}/step/${n}`, { method: "POST" });
  last = 0; poll();
}
$("#prev").onclick = () => go(step - 1);
$("#next").onclick = () => go(step + 1);
setInterval(poll, 2000); poll();
</script>
</body>
</html>

```

8. Setup and test

1. Build the circuit (Figure 1) and upload the firmware; the OLED should show "Ready".
2. Place the demo model files in static/models and start the Flask server.
3. Open the viewer on a phone/tablet with ?station=ST01 (HTTPS required for AR; use Caddy or nginx with a plant certificate).
4. Write UID 04A1B2C3 to a tag, or edit the DEMO dictionary to match one of your tags' UID (shown in the Serial Monitor at 115200 baud).
5. Tap the tag: LED flashes green, OLED shows WO-1042, the phone loads the model and step 1. Press the button to go to step 2.
6. Tap "View in AR" to place the model at scale on the real workbench.

9. Security and extensions

- Keep the station API key secret and rotate it; run the server inside the plant network or behind VPN/SSO. Never expose PLM credentials to the browser.
- Add per-step images/video, torque values from PLM characteristics, and operator sign-off written back to PLM (quality record).
- Replace polling with WebSocket/Server-Sent Events for instant updates; add an MQTT broker if you have many stations.
- Use image-target or marker AR (AR.js / 8th Wall / Vuforia) to anchor the model on the physical part instead of free placement.
- Cache models in a service worker so the app keeps working during Wi-Fi drop-outs.